

Project Proposal Form, Fall 2026
Institute of Information Technology (IIT)
University of Dhaka

Student's Name:			
Students' Roll:		Phone:	
Project Title: Design, Development and Evaluation of a Specification-Driven and Reusable Alumni Management Platform for a Resource-Constrained Association Short name: GHCAA — Govt. Haraganga College Alumni Association Platform Motivation: Alumni associations often depend on paper records, spreadsheets, informal messaging groups and manually maintained financial records. These practices make membership verification, payment tracking, communication, event coordination and governance-document management difficult. The Govt. Haraganga College Alumni Association needs a low-cost, maintainable and privacy-aware system of record that can be operated by volunteers. The platform must work without assuming enterprise-level staffing, recurring software licenses or live payment-gateway credentials. The project therefore focuses on: • reliable membership and identity records; • financial transparency and auditability; • privacy-aware directory access; • event and communication management; • publication of constitutional and governance documents; • mobile-first access; • single-maintainer sustainability; • specification-driven development and measurable testing. The project will also examine how far Association rules can be represented in software without allowing the software to replace the authority of the constitution or Election Commission. Project Description: GHCAA will be developed as a multi-client platform consisting of an ASP.NET Core Web API, an Angular web application and a Flutter mobile application. The project will follow a specification-driven development process. Requirements will be derived from stakeholder needs, existing manual processes, Association documents and software-engineering standards. Each important requirement will be traced to a business rule, implementation component, test and evaluation result where applicable. Main functional areas • Membership and identity: ○ Multi-step registration. ○ Academic and professional history. ○ Administrative approval. ○ Membership status and membership-number generation. ○ Digital membership cards and certificates. ○ Member profiles and privacy controls.			

- Searchable alumni directory.
- Secure identity and document handling.
- Accounting and payments
 - Membership fees, dues and donations.
 - Income and expense records.
 - Configurable payment instructions.
 - Manual payment-proof submission.
 - Administrative verification.
 - Receipts and audit history.
 - Optional future gateway adapters.

The initial operation does not require live gateway credentials. Members pay through administrator-configured bank or mobile-financial-service instructions and upload payment evidence for verification.

- Events and participation
 - Event publication.
 - Event registration and capacity control.
 - Waitlisting.
 - QR-based attendance.
 - Event tasks, budgets and expenses.
 - Member-submitted galleries with administrative approval.
- Communication and networking
 - News and notices.
 - Targeted member communication.
 - Real-time messaging and notifications.
 - Alumni search.
 - Job and mentorship features.
 - Contribution points and badges.
- Governance and administration
 - Constitution publication and version history.
 - Latest-version resolution.
 - Executive Committee records and terms.
 - Amendment proposals and eligibility-controlled voting.
 - Non-binding member polls.
 - Election voter-roll preparation.
 - Election-document and process support.
 - Administration, audit and organization configuration.

The system supports governance procedures but does not independently replace the Election Commission or claim to provide a legally sufficient electronic election system.

- Reusable platform foundation

The prototype already includes reusable platform capabilities:

- JWT authentication and password security;
- role-based authorisation;
- protected API and client routes;
- accounting, payment and ledger services;
- audit logging;
- file and document handling;
- notifications and SignalR messaging;
- organisation configuration;
- shared validation and error handling;
- reusable Angular controls;
- reusable Flutter widgets;
- database migration and deployment support;
- testing and CI/CD foundations.

Common frontend controls include page headers, search fields, filters, data tables, pagination, loading states, empty states, confirmation dialogs and shared form patterns.

The platform separates reusable technical services from organization-specific rules and configuration. This makes the foundation adaptable to similar alumni or membership organizations, while recognizing that each organization still requires its own rules, documents, branding and testing.

Optional AI-agent extension

The current assistant is rule-based and connects member questions to approved internal information and services. If time and evaluation evidence permit, a controlled AI-assisted agent may be added to help users:

- find policies and procedures;
- locate forms and documents;
- understand membership and payment steps;
- navigate the portal;
- search permitted directory information.

The agent would be advisory and tool restricted. It would not approve memberships, verify payments, modify accounting records, determine election eligibility or override privacy and authorization rules. If the required safety and evaluation evidence cannot be completed, the rule-based assistant will remain the delivered feature.

Development Approach

The project will use Design Science Research with incremental software development.

The development cycle will be:

- specify the requirement and acceptance condition;
- model the workflow, state or business rule;
- implement it across the required layers;
- test the implementation;
- trace the requirement to its evidence;
- evaluate the result and record limitations.

Architecture and design principles

The system will use:

- Clean Architecture;
- separation of domain, application, infrastructure and API concerns;
- dependency inversion;
- explicit application contracts and DTOs;
- typed API communication;
- dependency injection;
- role-based access control;
- validation at application and client boundaries;
- global soft-archive/query-filter behavior;
- centralized design tokens and reusable controls;
- configuration-driven organization branding and content;
- shared services instead of duplicated feature logic;
- incremental development with working software after each major stage.

Testing, Coverage and Quality Evaluation

The project will use:

- backend unit tests for services, validators and business rules;
- backend integration tests for database and API behavior;

- EF Core SQLite and in-memory test configurations;
- Angular unit tests using Vitest;
- Flutter widget and service tests;
- Playwright end-to-end tests;
- authentication, authorization and privacy tests;
- API contract and compatibility checks;
- selected visual regression tests;
- static analysis, type checking and build verification.

Coverage and quality evidence will include:

- backend statement and branch coverage using Coverlet;
- requirement coverage through the traceability matrix;
- targeted coverage of critical membership, accounting and governance rules;
- API and end-to-end test results;
- security-control assessment and threat-model review;
- usability task results;
- performance measurements;
- maintainability and deployment-cost assessment.

The project will report results by test suite and requirement category. It will not claim one combined coverage percentage unless such a figure is actually generated and verified.

The intended requirement target is that all **Must** requirements are traced to passing tests. Any requirement without sufficient evidence will be reported as uncovered or limited.

CI/CD and Deployment

The project will use Git and GitHub for incremental version control and GitHub Actions for automated validation.

The CI/CD process will include:

1. backend builds and formatting checks;
2. Angular type checking and tests;
3. Flutter analysis and tests;
4. API and integration tests;
5. Playwright end-to-end tests;
6. API contract snapshot checks;
7. integrated API-plus-SPA packaging;
8. Docker image creation;
9. controlled pre-production deployment.

The application is packaged as a deployable .NET and Angular service. PostgreSQL is used for hosted deployment, while SQLite is used for local development and tests. Database migrations are applied through the application's migration bootstrap process.

CI/CD and deployment are treated as part of the engineering solution because a volunteer-maintained system must be repeatable to build, test and deploy.

Strategy & Timeline:

The project Gantt chart is given below:

Week	Task No / Module	Description	Dependencies
1	Problem Analysis	Analyze the Association's current membership, payment, communication and governance processes.	None
2	Stakeholder and Document Review	Review stakeholder needs, constitution, election documents and existing manual forms.	1

3	Requirements Specification	Define functional requirements, quality requirements, scope and limitations.	2
4	Traceability and Acceptance Criteria	Map requirements to use cases, business rules, implementation areas and acceptance criteria.	3
5	Domain and Data Modelling	Model members, payments, events, governance records, users and related entities.	4
6	Architecture and API Design	Finalize Clean Architecture, API contracts, database design and client integration plan.	5
7	Authentication	Implement registration, login, password security, JWT sessions and account recovery flows.	6
8	Authorization and User Roles	Implement role-based access control, protected routes and administrative permissions.	7
9	Membership Management	Implement registration, academic records, professional records, approval and membership status.	7–8
10	Profile and Privacy	Implement profiles, privacy controls, secure documents and member preferences.	9
11	Directory and Digital Identity	Implement alumni search, privacy-aware directory results, digital ID cards and certificates.	10
12	Accounting Foundation	Implement fees, dues, donations, income, expenses and ledger structures.	6, 9
13	Manual Payment Workflow	Implement payment instructions, proof upload, administrative verification and payment status.	12
14	Financial Audit and Receipts	Implement audit history, financial summaries and downloadable payment receipts.	13
15	Event Management	Implement event creation, publication, registration, capacity and waitlisting.	9
16	Attendance and Gallery	Implement QR attendance, event tasks, budgets, expenses and moderated gallery submissions.	15
17	Communication and Messaging	Implement news, notices, notifications, targeted communication and real-time messaging.	8, 15
18	Governance Documents	Implement constitution publication, version history and Executive Committee records.	6, 9
19	Voting and Polls	Implement amendment voting, non-binding polls and eligibility controls.	18
20	Election Support	Implement voter-roll preparation, candidate information and election-document support.	18–19

21	Reusable Platform Refinement	Consolidate authentication, authorization, accounting, shared controls, organization configuration and common services.	7–20
22	Web and Mobile Integration	Complete Angular and Flutter integration against the shared API contracts.	21
23	Automated Testing	Run backend, web, mobile, API, integration and end-to-end tests. Add missing critical tests.	22
24	Coverage and Quality Evaluation	Collect coverage, requirement coverage, static-analysis results and contract-test evidence.	23
25	Security, Usability and Performance	Perform security review, privacy checks, usability evaluation and performance measurements.	23–24
26	Deployment and Final Documentation	Validate CI/CD, Docker packaging and deployment; complete report, presentation and future-work documentation.	25
	Optional AI-Agent Feasibility Extension	Add a controlled, advisory AI agent for policy lookup, portal navigation and approved information retrieval. The agent will use existing authentication, authorization and privacy controls and will not approve membership, verify payments, modify accounting records or determine election eligibility.	1-20

Languages or Tools to be used:

- Frontend: Angular, TypeScript, HTML, SCSS and Angular Signals
- Backend: ASP.NET Core 9 Web API, C# and Clean Architecture
- Mobile: Flutter and Dart
- Database: PostgreSQL for deployment; SQLite for local development and testing
- Real-time: ASP.NET Core SignalR
- Authentication: JWT, BCrypt and role-based authorisation
- Testing: NUnit, Moq, FluentAssertions, Vitest, Playwright, Flutter Test and integration tests
- Coverage: Coverlet and .NET test tooling
- Quality checks: .NET formatting, TypeScript type checking, Dart analysis and CI validation
- Version control: Git and GitHub
- CI/CD: GitHub Actions
- Packaging and deployment: Docker and hosted pre-production deployment
- IDE/Editor: Visual Studio Code, MS Visual Studio, Antigravity
- AI tools: AI Tools / Assistants: GitHub Copilot/Claude:

AI Tool Use Declaration:

☒ I intend to use AI coding/writing assistance tools for this project.

☐ I do not intend to use AI tools for this project.

If yes, tool(s) to be used:

Intended purpose(s) (e.g., boilerplate generation, debugging, documentation drafting, learning a new framework):

- framework and library learning;
- boilerplate generation;
- debugging;
- test suggestions;
- code review;
- documentation drafting;
- technical comparison;
- review of implementation decisions.

All AI-assisted output will be reviewed, tested and modified by the student. I'm responsible for the requirements, architecture, implementation, testing, security decisions and final report.

Declaration: I understand that, regardless of AI tool use, I am required to disclose such use in my project report, and I must be able to independently explain, defend, and modify any part of my code — including any AI-assisted sections — during the midterm and final presentations, live defense, and supervisor meetings throughout the semester. I understand that submitting AI-generated work I cannot explain or defend may result in a reduced grade or other academic consequences.

Student's Signature: _____

Date: _____

Project GitHub link: <https://github.com/shalin-rahman/GHCAA>

Note: Maintain a regular, incremental, and dated commit history throughout the semester. A single large commit submitted close to the deadline is treated as a red flag and may be investigated.

Supervisor's Name: Dr. Kazi Muheymin-Us-Sakib, Professor, Institute of Information Technology, University of Dhaka

Signature of the supervisor: _____

Date: _____

Before the Midterm Presentation:

I confirm that the progress is satisfactory and I am forwarding it for the midterm presentation.

I further confirm that the student has demonstrated independent understanding of the codebase — including any AI-assisted portions — and was able to explain and modify their prototype during our supervision meetings.

Signature of the supervisor: _____

Date: _____

Before the Final Presentation:

I confirm that the progress is satisfactory and I am forwarding it for the final presentation.

I further confirm that the student has consistently demonstrated understanding and ownership of their work — including any AI-assisted code — throughout the semester, based on regular supervision meetings and progress reviews.

Signature of the supervisor: _____

Date: _____

Proposal Presentation Feedback (if any):

Midterm Presentation Feedback (if any):

AI-Use Disclosure Review Notes (if any):

(For supervisor/evaluator use — note any discrepancy between disclosed AI use and demonstrated understanding.)